

HARARE INSTITUTE OF TECHNOLOGY

REG NO	**H250397M**
COURSE NO	**OOP**
DEGREE PROGRAM	**COMPUTER SCIENCE**
NAME	**LIONEL MUNYAWU**
LAB	

1. Difference between a Class and an Object. Where does each live in memory?

Class

A class is a blueprint or template that defines attributes (fields) and behaviors (methods).

Java
class Student {
String name;
void study() {}
}

Object

An object is an instance of a class.

Java
Student s = new Student();

Memory Location

Item	Memory Area
Class metadata	Method Area (Metaspace)
Object instance	Heap Memory
Reference variable (s)	Stack Memory (if local variable)

Mnemonic: Class = Blueprint, Object = Building.

---

2. Why does Java not allow multiple inheritance of classes? How do interfaces solve it?

Java prevents multiple inheritance of classes to avoid the **Diamond Problem**.

Example:

Java
```
class A {
void show() {}
}

class B extends A {}
class C extends A {}

// What if D inherits B and C?
class D extends B, C {} // Illegal
```

The compiler would not know which show() implementation to use.

Interface Solution

Java
```
interface Flyable {
void fly();
}

interface Swimmable {
void swim();
}

class Duck implements Flyable, Swimmable {
public void fly() {}
```

public void swim() {}
}

A class can implement many interfaces because interfaces define contracts rather than inherited state.

---

3. Overloading vs Overriding

Overloading (Compile-Time Polymorphism)

Same method name, different parameters.

Java
class Calculator {
int add(int a, int b) {
return a + b;
}

double add(double a, double b) {
return a + b;
}
}

Overriding (Run-Time Polymorphism)

Subclass provides a new implementation.

Java
class Animal {
void sound() {
System.out.println("Animal sound");
}

```
}

class Dog extends Animal {
@Override
void sound() {
System.out.println("Bark");
}
}
```

Summary

Overloading	Overriding
Same class	Parent-child classes
Different parameters	Same parameters
Compile time	Run time
Method signature changes	Method implementation changes

4. What happens if a superclass has no no-argument constructor and subclass omits super(...)?

Java automatically inserts:

Java
```
super();
```

If the superclass does not have a no-argument constructor, compilation fails.

Example

Java

```
class Parent {
Parent(String name) {}
}

class Child extends Parent {
Child() {
// super(); inserted automatically
}
}
```

Compiler error:

Plain Text

```
Constructor Parent() is undefined
```

Correct:

Java

```
class Child extends Parent {
Child() {
super("John");
}
}
```

5. When would you choose an Abstract Class over an Interface?

Choose an **Abstract Class** when:

- Classes share common state (fields)
- Classes share common code
- You want partial implementation

Java

```
abstract class Animal {
String name;

void eat() {
System.out.println("Eating");
}

abstract void makeSound();
}
```

Choose an **Interface** when:

- You need a contract
- Unrelated classes should share behavior
- Multiple inheritance of behavior is required

Java

```
interface Flyable {
void fly();
}
```

Quick Rule

"Is-a" relationship → Abstract Class

"Can-do" capability → Interface

6. Why must hashCode() be overridden whenever equals() is overridden?

Contract:

If two objects are equal according to equals(), they **must** have the same hash code.

Java
obj1.equals(obj2) == true

Then:

Java
obj1.hashCode() == obj2.hashCode()

Collections such as:

- HashMap
- HashSet
- Hashtable

use hash codes to locate objects.

Without overriding hashCode(), equal objects may go into different buckets.

Example:

Java
HashSet<Student> set = new HashSet<>();

You may get duplicate entries even though objects are logically equal.

7. Difference Between Checked and Unchecked Exceptions

Checked Exceptions

Checked by compiler.

Must be handled or declared.

Examples:

Java
IOException
SQLException
Java
```
try {
...
} catch(IOException e) {
}
```

Unchecked Exceptions

Occur during runtime.

Compiler does not force handling.

Examples:

Java
NullPointerException
ArithmeticException
Java
```
int x = 10 / 0;
```

Summary

Checked	Unchecked
Compile-time checking	Runtime checking
Must handle	Optional
Subclass of Exception	Subclass of RuntimeException

8. Why can't Generics use primitive types? What is Type Erasure?

Why not primitives?

Generics work with objects only.

Invalid:

Java
```
List<int> nums;
```

Valid:

Java
```
List<Integer> nums;
```

Java provides wrapper classes:

Primitive	Wrapper
int	Integer
double	Double
char	Character

Type Erasure

During compilation Java removes generic type information.

Java
```
List<String> names = new ArrayList<>();
```

becomes approximately:

Java
```
List names = new ArrayList();
```

at runtime.

Generics provide type safety during compilation, but runtime does not retain most generic type information.

9. What does final mean?

Final Variable

Cannot be reassigned.

Java
final int AGE = 20;

Final Method

Cannot be overridden.

Java
final void display() {}

Final Class

Cannot be inherited.

Java
final class Utility {}

Example:

Java
String

is a final class.

Memory Aid

Variable → Cannot change value

Method → Cannot change behavior

Class → Cannot extend

10. Explain Dynamic Method Dispatch. Why are fields and static methods not polymorphic?

Dynamic Method Dispatch means Java determines at runtime which overridden method to execute.

Java
```
Animal a = new Dog();
a.sound();
```

Output:

Plain Text
```
Bark
```

The actual object (Dog) decides.

Why fields are not polymorphic

Java
```
class A {
int x = 10;
}
```

```
class B extends A {
int x = 20;
}

A obj = new B();
System.out.println(obj.x);
```

Output:

```
Plain Text
10
```

Fields are resolved using the reference type.

Why static methods are not polymorphic

```
Java
class A {
static void show() {
System.out.println("A");
}
}

class B extends A {
static void show() {
System.out.println("B");
}
}

A obj = new B();
obj.show();
```

Output:

```
Plain Text
A
```

Static methods belong to the class, not the object.

11. What is a Functional Interface? How does a Lambda relate to it?

A functional interface contains exactly **one abstract method**.

Example:

Java
```
@FunctionalInterface
interface Greeting {
void sayHello();
}
```

Traditional implementation:

Java
```
Greeting g = new Greeting() {
public void sayHello() {
System.out.println("Hello");
}
};
```

Lambda version:

Java
```
Greeting g = () -> System.out.println("Hello");
```

Relationship

A lambda expression is a concise way to provide the implementation of a functional interface.

Examples:

- Runnable
- Comparator
- Predicate
- Consumer

12. How does Try-With-Resources differ from Try/Finally?

Traditional Try-Finally

Java

```
FileReader fr = new FileReader("file.txt");

try {
// use file
}
finally {
fr.close();
}
```

You must close resources manually.

Try-With-Resources

Java

```
try (FileReader fr = new FileReader("file.txt")) {
// use file
}
```

Java automatically closes the resource.

The resource must implement:

Java
AutoCloseable

Examples:

- FileReader
- Scanner
- BufferedReader
- Connection

Advantages

- Shorter code
- Safer
- Prevents resource leaks
- Automatic cleanup even when exceptions occur